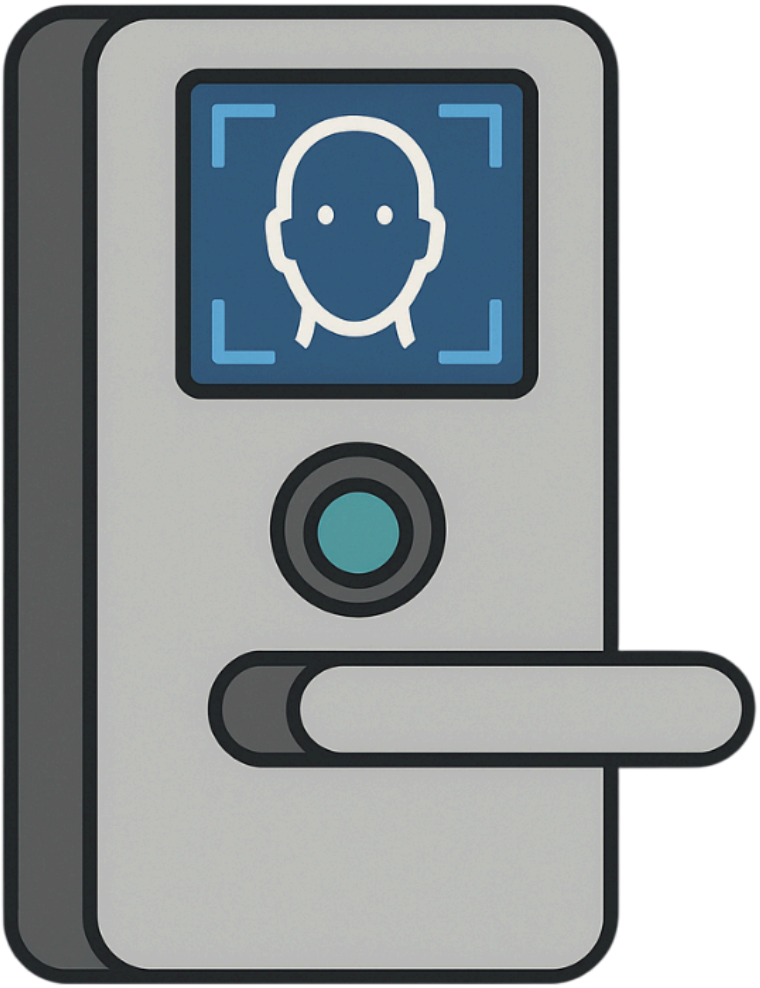
Verrou Biometrique



SPYCHER Adrian - BERNILLON Valentin

Mai 2025

HEPIA - Projet FPGA

1. Introduction....2
1.1 Contexte du projet....2
1.2 Objectif....2
2. Composants du système....3
2.1 Capteur de Proximité....3
2.2 Caméra....3
2.3 Verrou Solenoid 12V....4
3. Modules VHDL....4
3.1 Algorithme LBPH....4
3.2 Contrôleur VGA....7
4. Architecture globale....8
5. Designs et interfaces AXI....9
5.1 Timer....9
5.2 GPIO....9
5.2 IIC....9
5.2 Design VGA....9
6. Conclusion....10

1. Introduction

1.1 Contexte du projet

Ce projet, réalisé dans le cadre du cours de projet FPGA, vise à mettre en pratique les diverses connaissances étudiées dans ce cours, en les intégrant dans un projet complet intégrant capteurs, modules VHDL, et carte de développement FPGA Nexys Video.

L'idée de ce projet était de concevoir et implémenter un système de verrouillage biométrique, utilisant le FPGA de la Nexys Video, tout en utilisant le processeur softcore Microblaze pour la gestion générale et la gestion des interfaces AXI4-Lite, permettant ainsi la communication avec les différents périphériques. La solution proposée met en pratique l'utilisation de l'algorithme LBPH à l'aide d'une caméra pour de la reconnaissance faciale.

Avant toute avancée technique, la mise en place d'un cahier des charges complet et cohérent a permis de visualiser clairement les étapes de réalisations à venir, tout en établissant les divers composants matériels et périphériques externes nécessaires à la réalisation.

1.2 Objectif

L'objectif final de ce projet de verrou biométrique consiste à la création d'une solution complète utilisant la reconnaissance faciale pour contrôler le verrouillage / déverrouillage du système, activé en premier lieu par une détection de proximité.

Afin d'arriver à cette solution, les composants clés suivants ont étés déterminés, afin d'être intégré à la solution finale :

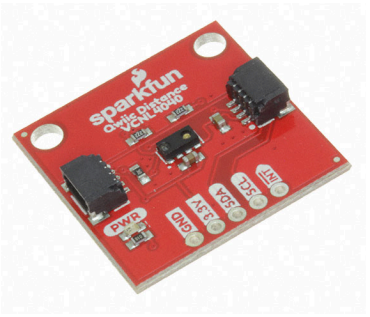
- Une caméra
- Un capteur de proximité
- Un verrou magnétique
- Un moniteur pour affichage vidéo
- Le FPGA Nexys Video

2. Composants du système

2.1 Capteur de Proximité

Un capteur de proximité est utilisé pour détecter l'approche d'un utilisateur. Ce capteur génère une interruption sur le FPGA lorsqu'une proximité est détectée. Il est configurable par IIC via une IP Xilinx. Ce capteur à été configuré de façon à envoyer une interruption lors de l'approche d'un objet en dessous d'un certain threshold, défini à 16 lx.





Afin d'utiliser ce capteur simplement, une carte de développement de chez Sparkfun a été utilisée. Celle-ci ajoute le hardware nécessaire au bon fonctionnement du capteur tel que les pull-up des lignes I2C.

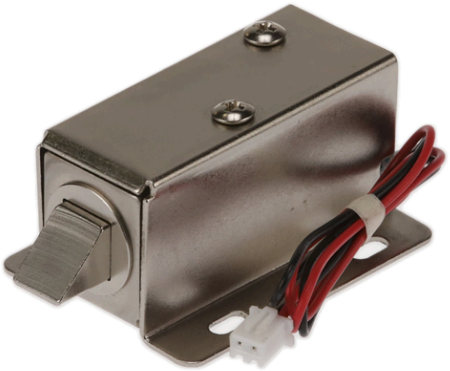
2.2 Caméra

La caméra utilisée pour la réalisation de ce projet est le module OV7670, un capteur d'image compact capable de capturer des images en format VGA (640×480) à une fréquence de 30 images par seconde. Sa configuration s'effectue via une interface I2C, qui permet de paramétrer les registres internes du capteur (résolution, format de sortie, horloge, …). Les images obtenues sont ensuite exploitées par l'algorithme de reconnaissance faciale, permettant ainsi d'activer ou désactiver le déverrouillage du locker.

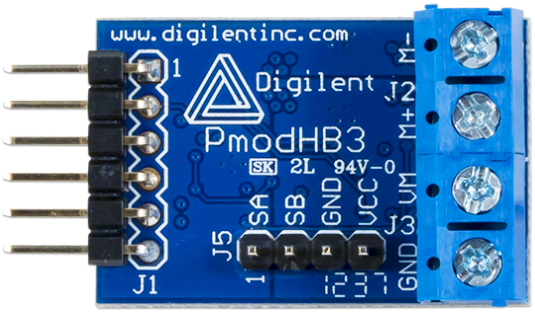


2.3 Verrou Solenoid 12V

Le verrou utilisé lors de ce projet, est un verrou à solenoid, c'est-à-dire, qu'il fonctionne grâce à un aimant. En effet, lorsque ce verrou recevra 12V, un aimant attira le loquet afin d'ouvrir le verrou.



Afin de pouvoir gérer l'alimentation de ce verrou, un Pmod HB3 a été utilisé. Il permet d'alimenter ou non grâce à un pont en H. Celui-ci est contrôlé via un GPIO.



3. Modules VHDL

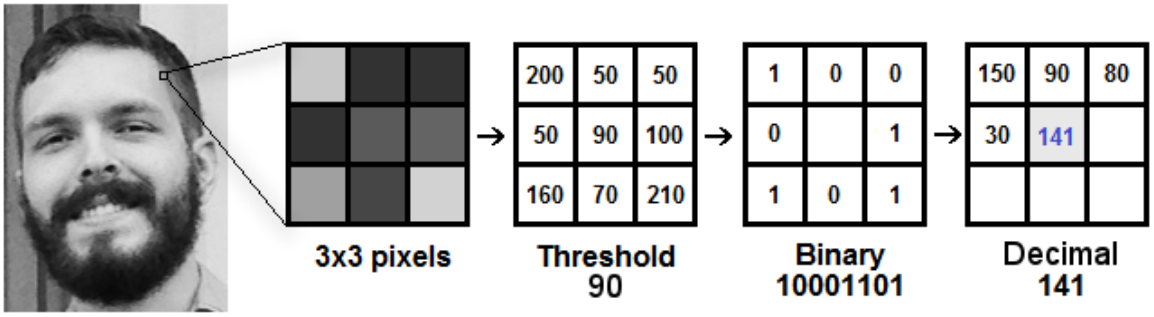
3.1 Algorithme LBPH

L'algorithme Local Binary Patterns Histograms (LBPH) est un algorithme de reconnaissance facial. Il peut être séparé en 3 étapes clefs :

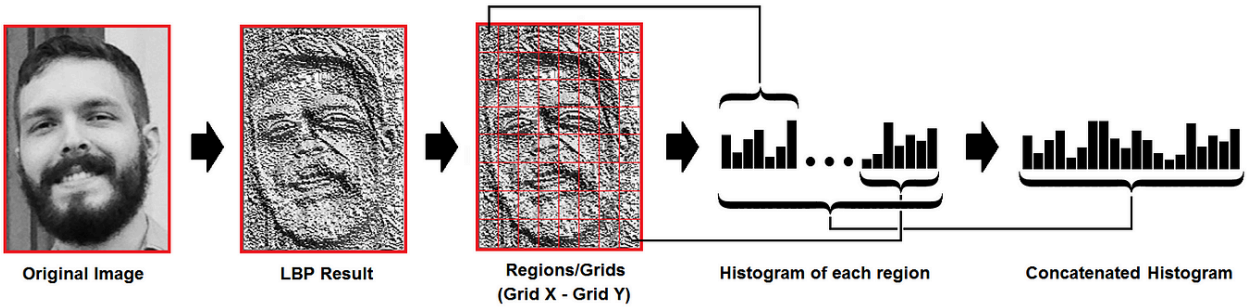
- une convolution permettant la détection de contour,
- la création d'un histogramme
- l'évaluation de la distance

Pour faire la détection de contour, l'algorithme passe sur chaque pixel en appliquant un filtre. Ce filtre consiste à comparer les pixels autour du pixel impliqué avec celui-ci. S'ils sont plus petits, ils prennent la valeur 0, autrement 1. Ces bits seront ensuite concaténés afin de donner de la valeur au pixel impliqué.

L'image ci-dessous, illustre ce processus :



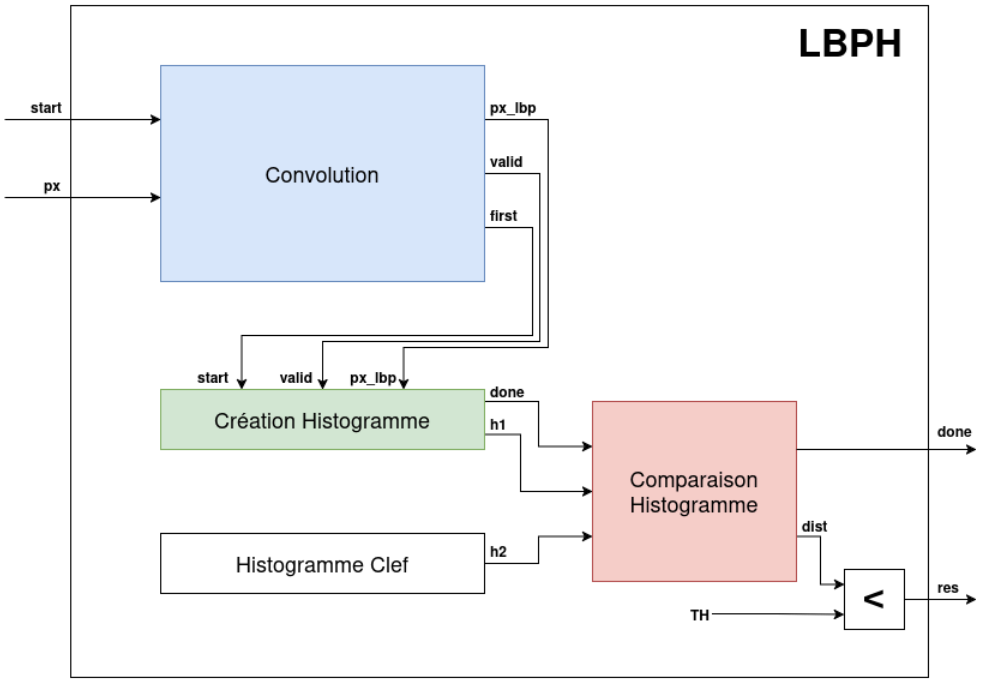
Pour créer l'histogramme, l'image filtrée serra séparée en plusieurs cellules. Chacune de ces cellules verront leurs histogrammes calculés. Ils seront ensuite tous concaténés afin d'avoir l'histogramme unique au visage.



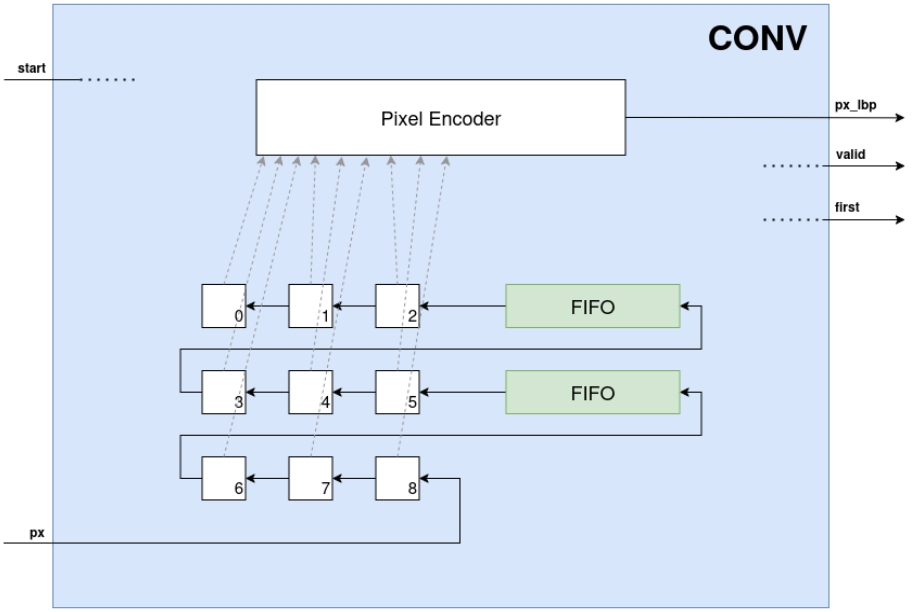
Afin de comparer deux histogrammes, il est nécessaire de faire la distance entre eux. Pour cela, nous avons choisi de faire la distance Euclidienne. Afin qu'elle soit réalisable en VHDL, nous avons enlevé la racine de l'équation.

$$D = \sum_{i=1}^{n} (hist1_i - hist2_i)^2$$

Cet algorithme a été implémenté dans un module VHDL, illustré par l'image ci-dessous :



Cette image représente un schéma bloc de l'algorithme LBPH, en couleur sont les 3 étapes clefs de l'algorithme; la convolution, la création de l'histogramme et l'évaluation de distance (comparaison). Il est important de noter que ce module reçoit un flux vidéo, et donc, les informations pixel par pixel. La convolution est plus détaillée dans l'image suivante :



Pour faire une convolution sur une image reçue pixel par pixel, il est nécessaire d'utiliser des FIFO afin de stocker et d'utiliser petit à petit les pixels reçus. Ces deux FIFO sont placés afin d'avoir dans 9 registre visible les pixels nécessaire pour effectuer le filtre.

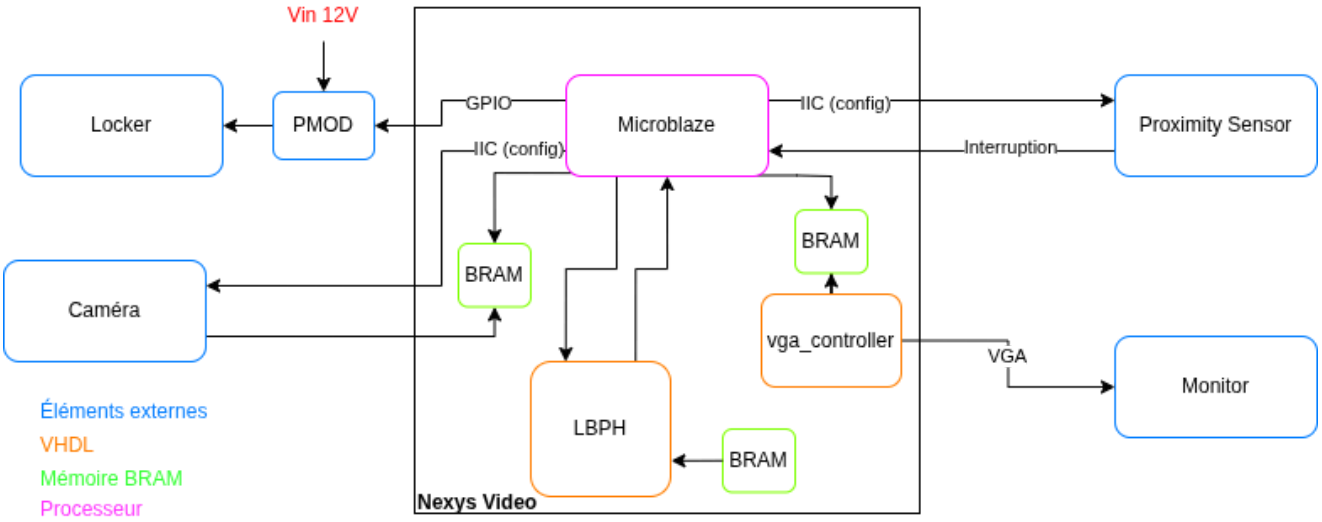
3.2 Contrôleur VGA

Le module du contrôleur VGA a été récupéré et adapté d’un ancien projet, avec pour principal objectif de valider le fonctionnement de la caméra et d'obtenir un retour des données capturées, afin d’assurer une base solide pour la mise en place de l’algorithme de reconnaissance faciale.

4. Architecture globale

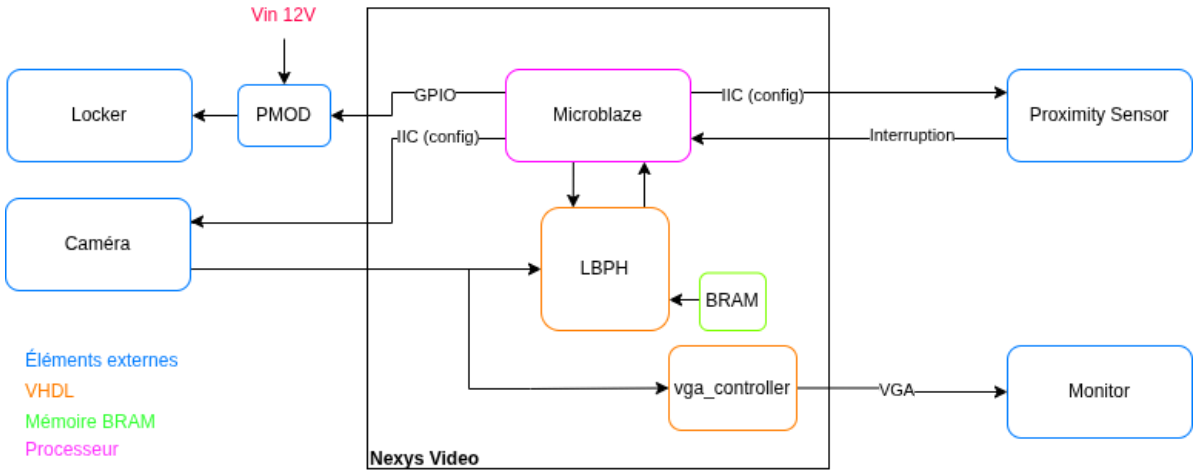
Une première version de l'architecture a été mise en place. Celle-ci intégrant toutes les fonctionnalités nécessaires à la communication avec les différents périphériques, en y incluant GPIO, IIC, interruptions, interconnectés avec les ports nécessaires en sortie. Cette version, fonctionnelle dans la théorie, a finalement été source de divers problèmes lors de sa réalisation.

Cette architecture avait pour objectif d'être au maximum paramétrable et configurable depuis Vitis, obligeant l'intégration de plusieurs BRAM et de communications par AXI pour des périphériques ou modules n'en n'ayant pas la nécessité absolue.



Cette deuxième version de l'architecture est similaire en plusieurs points à la première, notamment pour le capteur de proximité et le locker, mais diffère grandement pour la récupération des données de la caméra et l'affichage en sortie sur le VGA.

L'objectif de celle-ci est d'adapter directement l'architecture pour une liaison directe entre la caméra et l'affichage sur VGA, mais également avec le module LBPH s'occupant du traitement de l'image pour une reconnaissance faciale, limitant ainsi l'utilisation des BRAM.



Cette version n'aura malheureusement pas pu être implémentée dans son ensemble dans le temps imparti, mais présente globalement la solution la plus pertinente à ce jour.

5. Designs et interfaces AXI

Pour la réalisation de ce projet, une architecture et un design fiable et fonctionnel était impératif, c'est pourquoi chaque fonctionnalité a été implémentée puis testée avant d'être validée et intégrée dans le design final. Pour gérer correctement les communications entre les périphériques et le processeur, plusieurs interfaces AXI ont été mises en place.

5.1 Timer

Une implémentation du timer a été mise en place au début du projet, ceci afin de valider deux chose :

- La stabilité de l'architecture et son bon fonctionnement, sans bug, depuis l'environnement Vitis. (Permet des tests fiables de timings combiné avec l'utilisation des LEDs)
- Celui-ci a également servi de base au tests de fonctionnement des interruptions, qui seront utiles par la suite pour utiliser le capteur de proximité.

5.2 GPIO

Un premier GPIO a été configuré sur le bus AXI, afin de vérifier le fonctionnement des interruptions et autres fonctions de debug / timing directement sur les LEDs de la Nexys Video.

Un second module GPIO a été configuré, cette fois-ci dédié spécifiquement au capteur de proximité. Celui-ci permet de détecter lorsqu'une interruption est générée par le capteur, celle-ci est directement transmise au processeur pour un traitement adéquat dans le programme C.

5.2 IIC

L'implémentation de l'I2C était primordiale dans le cadre de ce projet. Celle-ci permettant de configurer deux des périphériques clés du fonctionnement du système, à savoir le capteur de proximité, et la caméra. L'interface IIC proposée par Xilinx permet la mise en place d'une communication par I2C directement par AXI, donnant accès à une configuration modulable depuis le code C dans l'environnement Vitis.

5.2 Design VGA

Pour obtenir l'affichage de la caméra sur le moniteur par VGA, à des fins de validation du fonctionnement, plusieurs stratégies ont été abordées et testées, ceci sur la base du contrôleur VGA réalisé dans le cadre du cours de FPGA durant le premier semestre de cette même année.

Un premier objectif d'adaptation a été d'avoir la possibilité de configurer tous les ports du contrôleur VGA depuis le programme C via le bus AXI, ceci pour obtenir une modularité complète (changement de moniteur, de résolution, etc..) sans avoir à reconstruire l'architecture et la plateforme à chaque fois. Pour ceci, le développement d'une IP personnalisée, permettant de configurer directement les ports du contrôleur VGA a été mise en place.
Cette solution a finalement été abandonnée, après avoir pris conscience qu'une clock programmable depuis Vitis serait également nécessaire pour atteindre l'objectif, ce qui correspondait à une charge de travail trop grande dans le temps dédié à ce projet.

Le deuxième objectif d'adaptation a été de mettre en place un framebuffer en BRAM, accessible directement depuis le bus AXI, afin de pouvoir décider de l'affichage sur le moniteur directement depuis le programme C. A terme, cette solution aurait permis d'afficher de façon flexible les données retournées par la caméra pour valider celles-ci.
Malheureusement, cette solution, bien que fonctionnelle en partie, comporte des erreurs de timing et de décalage, générant des erreurs d'affichage.

La solution finale, qui, par manque de temps, n'a pas pu être réalisée, aurait finalement été la plus simple, et consistait en une liaison directe entre les différents modules VHDL, pour rediriger les données de la caméra en entrée du contrôleur VGA. Ceci aurait permis un affichage direct et sans latence, tout en évitant de potentielles erreurs dans la mise en place de solution compliquée.

Avec plus de temps, c'est cette solution qui aurait été mise en place pour un affichage fonctionnel sur VGA.

6. Conclusion

La mise en place et la création d'un projet complet, dans le cadre du cours de projet FPGA, a été un exercice stimulant et intéressant, mais également difficile dans sa réalisation.
En effet, concevoir un cahier des charges complet, en y intégrant au mieux les attentes et les différentes solutions liées au projet final, tout en estimant correctement la faisabilité de celles-ci n'était pas une tâche facile.

Malgré les difficultés rencontrées, la division en "sous modules" des différentes fonctionnalités nécessaires au fonctionnement du Verrou Biométrique a permis d'obtenir des résultats concrets pour la plupart. Le fonctionnement du verrou et du capteur de proximité a pu être validé, l'algorithme de reconnaissance faciale a également pu être validé à l'aide de testbench et de simulation. Les problèmes rencontrés pour la configuration de la caméra et du contrôleur VGA ont permis d'acquérir une bonne maîtrise de l'architecture, permettant ainsi d'assurer sa stabilité.

Avec une meilleure gestion du temps, une solution complète et adaptée au FPGA aurait pu être proposée, permettant de trouver l'équilibre entre accélération matérielle et flexibilité logicielle.